

ECEN 4/5632 — Theory and Application of Digital Filters

Problem Set 4: FIR Design & Windowing (Lecture 7) + IIR Design & Bilinear Transform (Lecture 8)

Covers material from Lecture 7 (9/15/26) and Lecture 8 (9/17/26). Senior/first-year graduate level. Some problems require Python (NumPy/SciPy/PyTorch).

Part A — FIR Filters, Linear Phase, and Windowing

A1. Linear phase and group delay (Core)

A causal FIR filter has taps $h[n]$, $n = 0, \dots, N - 1$, satisfying the symmetry condition $h[n] = h[N - 1 - n]$.

(a) Starting from $H(e^{j\omega}) = \sum_{n=0}^{N-1} h[n]e^{-j\omega n}$, show that $H(e^{j\omega})$ can be written as $e^{-j\omega(N-1)/2}A(\omega)$ where $A(\omega)$ is real-valued.

(b) Using the result of (a), derive the group delay $\tau_g(\omega) = -\frac{d}{d\omega} \angle H(e^{j\omega})$ and show it is constant whenever $A(\omega) > 0$.

(c) For $N = 63$ taps, what is the group delay in samples? If the sampling rate is 8 kHz, what is the group delay in milliseconds?

(d) Explain, in one or two sentences, why "linear phase" does **not** mean "zero delay." Give a concrete signal-processing scenario (unrelated to the lecture's MFCC example) where this distinction matters.

A2. Deriving the ideal lowpass impulse response (Core)

The ideal (brick-wall) lowpass filter is defined by

$$H_d(e^{j\omega}) = \begin{cases} 1, & |\omega| \leq \omega_c \\ 0, & \omega_c < |\omega| \leq \pi \end{cases}$$

(a) Compute the inverse DTFT $h_d[n]$ and show your work for the case $n \neq 0$ and the limiting case $n = 0$.

(b) Verify that $h_d[n]$ is even in n (i.e., $h_d[n] = h_d[-n]$), and explain why this is required for a zero-phase (noncausal, doubly-infinite) filter.

(c) If $\omega_c = \pi/2$, what is $h_d[0]$? What fraction of the total "energy" of $h_d[n]$ (informally, $|h_d[0]|^2$ relative to $|h_d[n]|^2$ at $n = \pm 1$) is concentrated at the origin?

A3. Truncation, shifting, and the Gibbs phenomenon (Core)

(a) Starting from $h_d[n]$ in A2, write the expression for the ideal impulse response **centered and shifted** to produce a causal FIR of order M (i.e., $M + 1$ taps). What is the resulting group delay?

(b) For $N = 101$ taps and $\omega_c = 0.3\pi$, use Python to compute and plot the rectangularly truncated filter's magnitude response $|H(e^{j\omega})|$ in dB. Report the approximate passband overshoot (in dB, relative to 0 dB) near the cutoff.

(c) Repeat for $N = 21$ and $N = 201$. Does the *peak* overshoot (in dB, not the width of the ripple region) change appreciably with N ? State the Gibbs phenomenon result this illustrates, including the approximate asymptotic overshoot percentage ($\sim 9\%$) if you can find/derive it.

(d) A student claims: "If I just use enough taps, rectangular windowing will eventually meet any stopband attenuation spec." Explain why this is false, referencing your result from (c).

A4. Window comparison and filter-length budgeting (Core)

Use the table below (also given in Lecture 7) for approximate window properties:

Window	First sidelobe (dB)	Main-lobe width
Rectangular	-13	$4\pi/N$
Bartlett	-26	$8\pi/N$
Hann	-31	$8\pi/N$
Hamming	-43	$8\pi/N$
Blackman	-58	$12\pi/N$

You are asked to design a linear-phase FIR lowpass filter with:

- Passband edge $\omega_p = 0.35\pi$, stopband edge $\omega_s = 0.45\pi$
- Minimum stopband attenuation of 50 dB

(a) Which window(s) from the table can plausibly meet the 50 dB stopband requirement? Which cannot, regardless of length? Justify using the first-sidelobe column.

(b) Using the main-lobe-width approximation $\Delta\omega \approx$ (transition width), estimate the minimum number of taps N needed with your chosen window to achieve a transition band no wider than $\omega_s - \omega_p = 0.1\pi$.

(c) Compute the resulting group delay in samples for your design.

(d) Design the filter in Python (`scipy.signal.firwin` or manual windowing) and verify with `freqz` that your stopband attenuation target is met. Attach your magnitude response plot (in dB) with the specification lines marked.

A5. Window method as frequency-domain convolution (Advanced)

The window method result $H_w(e^{j\omega}) = \frac{1}{2\pi} H_d(e^{j\omega}) * W(e^{j\omega})$ is a periodic convolution.

(a) Explain intuitively why a *narrower* main lobe of $W(e^{j\omega})$ leads to a *sharper* transition band in $H_w(e^{j\omega})$.

(b) Explain intuitively why *lower* sidelobes in $W(e^{j\omega})$ lead to *lower* ripple in $H_w(e^{j\omega})$, and why these two goals (narrow main lobe, low sidelobes) are in tension for a fixed window length N . (Hint: think about the time-domain taper's smoothness versus its duration.)

A6. Conv1d and the sliding dot product (Core)

Given the DSP convolution sum $y_{\text{DSP}}[n] = \sum_k h[k] x[n - k]$ and PyTorch's cross-correlation convention $y_{\text{torch}}[n] = \sum_k w[k] x[n + k]$:

(a) Derive the relationship $w[k] = h[M - k]$ (where h has $M + 1$ taps, indices $0, \dots, M$) that makes `F.conv1d` reproduce the textbook causal FIR convolution, and state the required left-padding amount.

(b) Implement both the NumPy convolution (`np.convolve` or a manual sliding sum) and the equivalent `F.conv1d` computation for a Hamming-windowed lowpass filter of your choice acting on a test signal of your choice (e.g., a chirp or noisy sinusoid). Report the maximum absolute difference between the two outputs (you should get something on the order of 10^{-6} – 10^{-7} in float32).

(c) A colleague says "PyTorch's Conv1d does convolution backwards, so it's a bug." Give a one-paragraph rebuttal, using the ML viewpoint discussed in lecture.

A7. Tensor shapes (Core, short answer)

You have 64 three-second audio clips, sampled at 44.1 kHz, in stereo (2 channels), which you want to filter with a bank of 16 fixed FIR kernels of length 31 using a single `Conv1d` call.

(a) What is the shape of the input tensor (`x`) (using PyTorch's (N, C, L) convention)?

(b) What shape must the `Conv1d` weight tensor have (PyTorch convention is `(out_channels, in_channels/groups, kernel_size)`) if you want each of the 16 kernels applied independently to **each** of the 2 input channels, producing 32 output channels? What value of `groups` would you use, and why?

A8. Conceptual / minute-paper synthesis (Short answer, Core)

In 3–5 sentences, answer the Lecture 7 minute-paper prompt: "**What does a neural 1-D convolution add to the FIR operation itself?**" Your answer should explicitly mention (i) what stays the same mathematically, and (ii) what is new when weights become learnable.

Part B — IIR Filter Design and the Bilinear Transform

B1. Choosing an analog prototype (Core)

For each scenario below, choose the most appropriate analog prototype (Butterworth, Chebyshev I, Chebyshev II, or elliptic) and justify your choice in one sentence using the passband/stopband ripple behavior:

(a) A calibration filter for a precision sensor, where passband amplitude flatness is critical and a somewhat gentle transition is acceptable.

(b) An application where minimizing filter order is the primary goal, ripple is tolerable in **both**

bands, and the transition must be as narrow as possible.

(c) An application where the passband must be perfectly flat (no ripple at all is tolerable there), but some ripple in the stopband is acceptable if it reduces order.

(d) An application where a small, controlled amount of passband ripple is an acceptable trade for a narrower transition than Butterworth can provide at the same order.

B2. Butterworth order and cutoff behavior (Core)

The Butterworth magnitude-squared response is

$$|H_c(j\Omega)|^2 = \frac{1}{1 + (\Omega/\Omega_c)^{2N}}$$

(a) Show that $|H_c(j\Omega_c)| = 1/\sqrt{2}$ regardless of N , and convert this to dB.

(b) Derive the standard Butterworth order formula that guarantees a passband attenuation $\leq A_p$ dB at Ω_p and a stopband attenuation $\geq A_s$ dB at Ω_s :

$$N \geq \frac{\log_{10} \left[\frac{10^{A_s/10} - 1}{10^{A_p/10} - 1} \right]}{2 \log_{10}(\Omega_s/\Omega_p)}$$

(Hints: write $|H_c(j\Omega_p)|^2$ and $|H_c(j\Omega_s)|^2$ in terms of N , form a ratio, and solve for N .)

(c) For $A_p = 1$ dB at $\Omega_p = 1$ rad/s and $A_s = 40$ dB at $\Omega_s = 3$ rad/s, compute the minimum required order N . Round up to the nearest integer and explain why you round up rather than down.

B3. The bilinear transform maps the LHP to the unit disk (Core)

The bilinear transform is $s = \frac{2}{T} \frac{1 - z^{-1}}{1 + z^{-1}}$, equivalently $z = \frac{1 + sT/2}{1 - sT/2}$.

(a) Let $s = \sigma + j\Omega$. Show algebraically that $\sigma < 0 \implies |z| < 1$, $\sigma = 0 \implies |z| = 1$, and $\sigma > 0 \implies |z| > 1$.

(b) Explain in your own words why property (a) is essential for using this substitution to convert a stable analog filter design into a stable digital filter.

(c) Contrast this with the impulse-invariance mapping $z = e^{sT}$. Both preserve stability — what problem does impulse invariance have that the bilinear transform avoids? What problem does the bilinear transform introduce instead?

B4. Deriving the frequency warping relation (Core)

(a) Substitute $s = j\Omega$ and $z = e^{j\omega}$ into the bilinear transform and show that

$$\Omega = \frac{2}{T} \tan\left(\frac{\omega}{2}\right), \quad \omega = 2 \tan^{-1}\left(\frac{\Omega T}{2}\right)$$

(b) Show that for small ω , $\Omega \approx \omega/T$ (i.e., the mapping is approximately linear near DC).

(c) What happens to Ω as $\omega \rightarrow \pi^-$? Explain the practical consequence for a digital filter's stopband edge if a designer forgets to prewarp.

B5. Prewarping design exercise (Core)

You need a digital Butterworth lowpass filter with $T = 1$ (i.e., $f_s = 1$ Hz, normalized) and a digital passband edge at $\omega_p = 0.3\pi$ where the response should be at -1 dB, and a digital stopband edge at $\omega_s = 0.6\pi$ where the response should be at least -30 dB down.

(a) Prewarp both edges to analog frequencies Ω_p and Ω_s using the formula from B4.

(b) Using your formula from B2(b), compute the required Butterworth order N .

(c) In Python, design the analog Butterworth prototype at the prewarped Ω_p (or Ω_c , your choice of convention) using `scipy.signal.butter(..., analog=True)`, then apply `scipy.signal.bilinear` (or `bilinear_zpk`) to obtain the digital filter. Plot the digital magnitude response and confirm the -1 dB and -30 dB points land at (approximately) 0.3π and 0.6π .

(d) Repeat the design **without** prewarping (i.e., using $\Omega_p = \omega_p$, $\Omega_s = \omega_s$ directly as analog frequencies) and show on the same plot how far the resulting digital cutoff misses the intended target. Quantify the error in normalized frequency.

B6. Group delay of an IIR filter (Core)

For the Butterworth IIR filter designed in B5(c):

(a) Compute and plot the group delay $\tau_g(\omega)$ over $0 \leq \omega \leq \pi$ using `scipy.signal.group_delay`.

(b) Contrast this plot qualitatively with the (flat) group delay of a comparable linear-phase FIR filter from Part A. In 2–3 sentences, describe a downstream ML task where this frequency-dependent group delay would matter, and one where it would not.

B7. Forward-backward (zero-phase) filtering (Core)

Let $H(e^{j\omega})$ be the transfer function of a real-coefficient causal IIR filter.

(a) Show that filtering a signal once forward and once backward (as in `scipy.signal.filtfilt`) produces a net frequency response $H_{fb}(e^{j\omega}) = |H(e^{j\omega})|^2$, and explain why the net phase is exactly zero.

(b) If the causal filter has -3 dB at ω_c , what is the forward-backward filter's attenuation (in dB) at that same ω_c ? Generalize: if you need a specific -3 dB point *after* forward-backward filtering, how should you adjust your single-pass design target?

(c) List two reasons `filtfilt`-style zero-phase filtering **cannot** be used in a real-time streaming deployment, and one reason it is nonetheless attractive for **offline** dataset preprocessing in an ML pipeline.

(d) In Python, generate a noisy transient signal (e.g., a Gaussian bump plus high-frequency noise), filter it with (i) your causal IIR lowpass from B5, and (ii) `filtfilt` using the same filter

coefficients. Plot both outputs against the clean reference signal and comment on the timing alignment of the transient peak in each case.

B8. Train/serve skew (Advanced, short essay)

Suppose a research team preprocesses their entire training dataset offline using `filtfilt` with an IIR lowpass filter to denoise sensor signals before training a classifier, then deploys the trained model in a real-time embedded system using a causal (single-pass) IIR filter with the *same* coefficients, reasoning that "it's the same filter."

(a) Identify at least three concrete ways the deployed model's input distribution will differ from what it saw during training.

(b) Propose one design change to either the training pipeline or the deployment pipeline that would reduce or eliminate this train/serve skew, and explain the trade-off it introduces.

B9. FIR vs. IIR vs. learned front end (Synthesis, Core)

Complete the following comparison in your own words (do not just copy the lecture table) for an application of your choosing (e.g., ECG denoising, keyword spotting, vibration monitoring):

Design choice	Strength	Cost/risk	Why it fits (or doesn't) your chosen application
Linear-phase FIR			
IIR (causal)			
IIR with <code>filtfilt</code>			
Learned Conv1d			

B10. Minute-paper synthesis (Short answer, Core)

In 3–5 sentences, answer the Lecture 8 minute-paper prompt: "**Why can a zero-phase offline preprocessing pipeline give misleading confidence about a streaming deployment?**"

Part C — Integrative Problem (Advanced, suggested for graduate students / bonus for undergraduates)

You are building an offline-trained, online-deployed system that classifies short mechanical vibration bursts as "normal" or "fault." Sensor data is sampled at 2 kHz. Known line-frequency interference sits at 60 Hz and its harmonics; the fault signature of interest lives in the 100–400 Hz band and its *onset timing relative to a trigger event* is an important feature.

(a) Would you recommend a linear-phase FIR notch/bandpass, an IIR notch/bandpass, or a learned Conv1d front end for the **deployed, real-time** stage? Justify using at least two concrete arguments from Parts A and B (e.g., group delay, order/latency trade-off, adaptability).

(b) Sketch (in words or a block diagram) a preprocessing pipeline that could be used **during training only** that would *not* be safe to reuse verbatim at deployment, and explain precisely

why.

(c) Propose a validation experiment you could run (in Python, at least in pseudocode) to detect whether train/serve skew of this kind is hurting your model's deployed accuracy before you ship it.

Submission Notes

- For all problems requiring Python, submit code (`.py` or notebook), one clearly labeled plot per sub-part, and a short written interpretation (2-4 sentences) beneath each plot.
- Show all derivation steps for algebraic/analytical problems (A1, A2, A3a, B2, B3, B4) — final answers without work receive partial credit only.
- Graduate students (5632) must complete Part C in full; undergraduates (4632) may complete Part C for up to 10% extra credit.